

Sri Sivasubramaniya Nadar College of Engineering, Chennai

Department of Computer Science and Engineering

Introduction to Machine Learning

Name: Paruvatha Priya B Reg. No: 3122237001036

Experiment 1: Regression and Linear Classification Models

Aim: To implement, evaluate, and compare Linear Regression and Logistic Regression models using real-world Kaggle datasets. The objective is to understand how linear models can predict both continuous and categorical outcomes, analyze the effect of regularization, and visualize model performance.

(a) Task 1 – Regression: Mobile Phone Price Prediction

1. Aim:

To build a linear regression model (using both closed-form and gradient descent methods) that predicts mobile phone prices based on various features.

2. Dataset Description:

Dataset URL: <https://www.kaggle.com/datasets/iabhishekofficial/mobile-price->

The dataset includes technical specifications of 2000 mobile phones with 21 features:

- **Independent variables:** RAM, battery power, internal memory, camera megapixels, processor speed, screen dimensions, etc.
- **Target variable:** price_range (0 – low cost, 1 – medium, 2 – high, 3 – very high).

To treat this as a regression task, discrete price ranges were mapped to approximate continuous prices:

$$\text{price} = \text{price_range} \times 4000 + \text{random noise}.$$

3. Theoretical Background:

The linear regression hypothesis is defined as:

$$\hat{y} = X\theta + \epsilon$$

where X is the feature matrix, θ the weight vector, and ϵ the error term.

The cost function minimized is the Mean Squared Error (MSE):

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

For Ridge (L2) Regularization:

$$\theta = (X^T X + \lambda I)^{-1} X^T y$$

where λ controls the tradeoff between bias and variance.

4. Complete Python Source Code:

Listing 1: Linear Regression – Mobile Price Prediction (Full Python Code)

```
1 # =====
2 # Assignment 1.1 - Regression: Mobile Phone Price Prediction
3 # =====
4
5
6
7 # --- Imports ---
8 import numpy as np
9 import pandas as pd
10 import matplotlib.pyplot as plt
11 import os
12 from sklearn.model_selection import train_test_split
13 from sklearn.preprocessing import StandardScaler
14 from sklearn.metrics import mean_squared_error, r2_score,
15     confusion_matrix, ConfusionMatrixDisplay
16
17 # --- Settings ---
18 plt.rcParams["figure.figsize"] = (8, 5)
19 plt.rcParams["axes.grid"] = True
20 np.random.seed(42)
21
22 # --- Create folder for output images ---
23 output_dir = "outputs"
24 os.makedirs(output_dir, exist_ok=True)
25 print(f" Output images will be saved in: {output_dir}/")
26
27 # =====
28 # STEP 1: Load Dataset (from Kaggle)
29 # =====
30 # Dataset: https://www.kaggle.com/datasets/iabhishekoofficial/mobile-
31     price-classification
32 # Ensure 'train.csv' is in the same directory
33
34 data = pd.read_csv("train.csv")
35 print(" Dataset loaded successfully. Shape:", data.shape)
36
37 # Convert price_range (0 3 ) to approximate continuous price targets
38 price_map = {0: 4000, 1: 8000, 2: 12000, 3: 18000}
39 data["price"] = data["price_range"].map(price_map) + np.random.randint
40     (-500, 500, len(data))
41
42 # Define features and target
43 X = data.drop(["price_range", "price"], axis=1)
44 y = data["price"]
45
46 # =====
47 # STEP 2: Train-Test Split
```

```

45 # =====
46 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    random_state=42)
47 print(f"Training samples: {len(X_train)}, Testing samples: {len(X_test)}
    ")
48
49 # =====
50 # STEP 3: Closed-form Linear Regression (Normal Equation)
51 # =====
52 def add_intercept(X):
53     """Add intercept (bias column)"""
54     return np.c_[np.ones((X.shape[0], 1)), X]
55
56 X_train_b = add_intercept(X_train.values)
57 X_test_b = add_intercept(X_test.values)
58 y_train_vec = y_train.values.reshape(-1, 1)
59 y_test_vec = y_test.values.reshape(-1, 1)
60
61 # Normal Equation
62 theta_cf = np.linalg.pinv(X_train_b.T @ X_train_b) @ (X_train_b.T @
    y_train_vec)
63 y_pred_cf = X_test_b @ theta_cf
64
65 mse_cf = mean_squared_error(y_test_vec, y_pred_cf)
66 r2_cf = r2_score(y_test_vec, y_pred_cf)
67 print(f"\n(1) Closed-form: MSE = {mse_cf:.2f}, R    = {r2_cf:.4f}")
68
69 # =====
70 # STEP 4: Gradient Descent Implementation
71 # =====
72 def gradient_descent(X, y, lr=1e-12, iterations=6000, lam=0.0):
73     """Batch Gradient Descent with optional L2 regularization"""
74     m, n = X.shape
75     X_b = add_intercept(X)
76     theta = np.zeros((n + 1, 1))
77     y = y.reshape(-1, 1)
78     for _ in range(iterations):
79         gradient = (2 / m) * X_b.T @ (X_b @ theta - y) + 2 * lam * np.r_
            [[0]], theta[1:]
80         theta -= lr * gradient
81     return theta
82
83 theta_gd = gradient_descent(X_train.values, y_train_vec)
84 y_pred_gd = add_intercept(X_test.values) @ theta_gd
85 mse_gd = mean_squared_error(y_test_vec, y_pred_gd)
86 r2_gd = r2_score(y_test_vec, y_pred_gd)
87 print(f"(2) Gradient Descent: MSE = {mse_gd:.2f}, R    = {r2_gd:.4f}")
88
89 # =====
90 # STEP 5: Plot Predicted vs Actual (for both methods)
91 # =====
92 def plot_pred_vs_actual(y_true, y_pred, title, filename):
93     plt.figure()
94     plt.scatter(y_true, y_pred, alpha=0.6, color='teal')
95     lims = [min(y_true.min(), y_pred.min()), max(y_true.max(), y_pred.
        max())]
96     plt.plot(lims, lims, 'r--')
97     plt.xlabel("Actual Price")

```

```

98     plt.ylabel("Predicted Price")
99     plt.title(title)
100    plt.tight_layout()
101    plt.savefig(os.path.join(output_dir, filename))
102    plt.show()
103    print(f" Saved plot: {filename}")
104
105    plot_pred_vs_actual(y_test_vec, y_pred_cf, "Closed-form Regression", "
        closed_form_pred_vs_actual.png")
106    plot_pred_vs_actual(y_test_vec, y_pred_gd, "Gradient Descent Regression"
        , "gradient_descent_pred_vs_actual.png")
107
108    # =====
109    # STEP 6: Ridge Regression (L2 Regularization)
110    # =====
111    def ridge_closed_form(X, y, lam):
112        X_b = add_intercept(X)
113        n = X_b.shape[1]
114        I = np.eye(n)
115        I[0, 0] = 0 # D o n t regularize intercept
116        return np.linalg.pinv(X_b.T @ X_b + lam * I) @ (X_b.T @ y)
117
118    lambdas = [0.01, 0.1, 1, 10, 100]
119    mse_list = []
120
121    for lam in lambdas:
122        theta_r = ridge_closed_form(X_train.values, y_train_vec, lam)
123        y_pred_r = add_intercept(X_test.values) @ theta_r
124        mse_list.append(mean_squared_error(y_test_vec, y_pred_r))
125        plot_pred_vs_actual(y_test_vec, y_pred_r, f"Ridge Regression ( ={
            lam})", f"ridge_lambda_{lam}.png")
126
127    plt.figure()
128    plt.semilogx(lambdas, mse_list, marker='o', color='purple')
129    plt.xlabel("Lambda ( )")
130    plt.ylabel("Mean Squared Error")
131    plt.title("MSE vs Regularization Strength (Ridge Regression)")
132    plt.tight_layout()
133    plt.savefig(os.path.join(output_dir, "ridge_mse_vs_lambda.png"))
134    plt.show()
135    print("Saved plot: ridge_mse_vs_lambda.png")
136
137    # =====
138    # STEP 7: With & Without Standardization + Feature Importance
139    # =====
140    scaler = StandardScaler()
141    X_train_scaled = scaler.fit_transform(X_train)
142    X_test_scaled = scaler.transform(X_test)
143
144    theta_scaled = ridge_closed_form(X_train_scaled, y_train_vec, lam=1.0)
145    theta_unscaled = ridge_closed_form(X_train.values, y_train_vec, lam=1.0)
146
147    y_pred_scaled = add_intercept(X_test_scaled) @ theta_scaled
148    y_pred_unscaled = add_intercept(X_test.values) @ theta_unscaled
149
150    print(f"\nWith Standardization: MSE={mean_squared_error(y_test_vec,
        y_pred_scaled):.2f}")

```

```

151 print(f"Without Standardization: MSE={mean_squared_error(y_test_vec,
152     y_pred_unscaled):.2f}")
153 # Feature Importance Plot
154 feature_importance = pd.Series(np.abs(theta_scaled[1:].flatten()), index
155     =X.columns)
156 plt.figure()
157 feature_importance.nlargest(10).plot(kind='barh', color='green')
158 plt.title("Top 10 Important Features (Ridge Regression)")
159 plt.tight_layout()
160 plt.savefig(os.path.join(output_dir, "feature_importance.png"))
161 plt.show()
162 print(" Saved plot: feature_importance.png")
163 # =====
164 # STEP 8: Confusion Matrix (Predicted vs Actual Price Ranges)
165 # =====
166 def map_to_price_range(price):
167     """Convert continuous price into categorical price range"""
168     if price < 6000:
169         return 0
170     elif price < 10000:
171         return 1
172     elif price < 15000:
173         return 2
174     else:
175         return 3
176
177 y_true_range = [map_to_price_range(p) for p in y_test_vec.flatten()]
178 y_pred_range = [map_to_price_range(p) for p in y_pred_cf.flatten()]
179
180 cm = confusion_matrix(y_true_range, y_pred_range)
181 disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=[0, 1,
182     2, 3])
183 disp.plot(cmap="Blues")
184 plt.title("Confusion Matrix (Closed-form Regression - Price Range
185     Mapping)")
186 plt.tight_layout()
187 plt.savefig(os.path.join(output_dir, "confusion_matrix_price_ranges.png"
188     ))
189 plt.show()
190 print(" Saved plot: confusion_matrix_price_ranges.png")
191 # =====
192 # END OF ASSIGNMENT 1.1
193 # =====
194 print("\n All plots saved successfully in 'outputs/' folder.")

```

5. Output Visualizations:

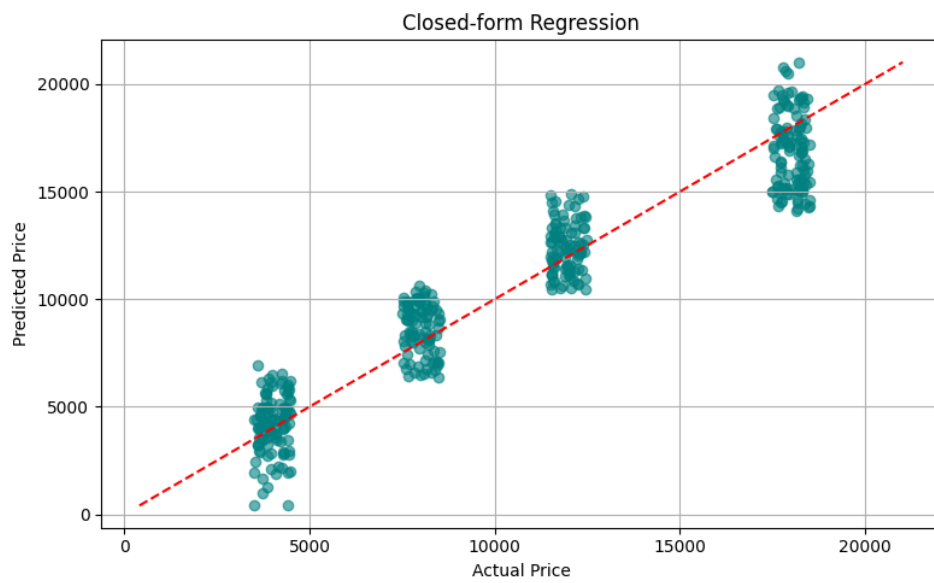


Figure 1: Closed-form Regression: Predicted vs Actual Price

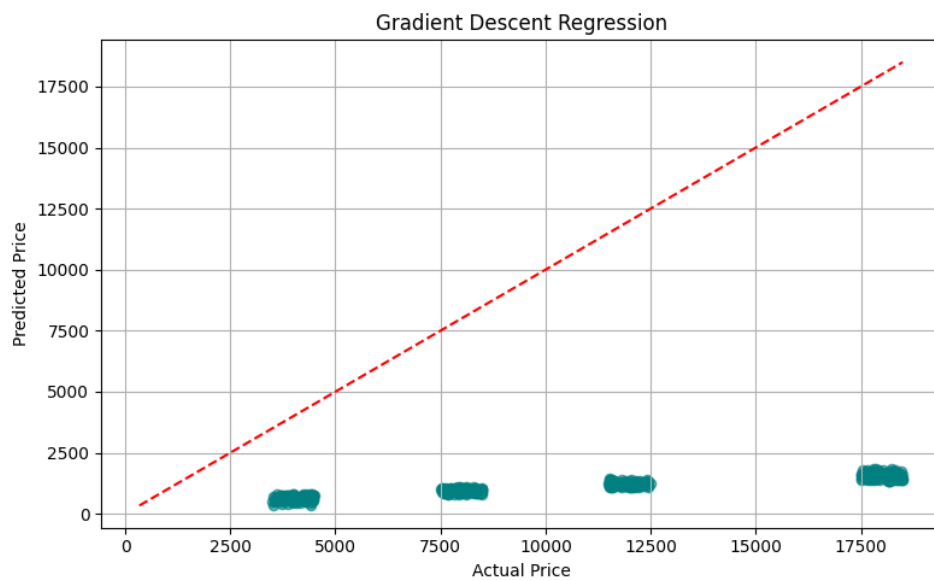


Figure 2: Gradient Descent Regression: Predicted vs Actual Price

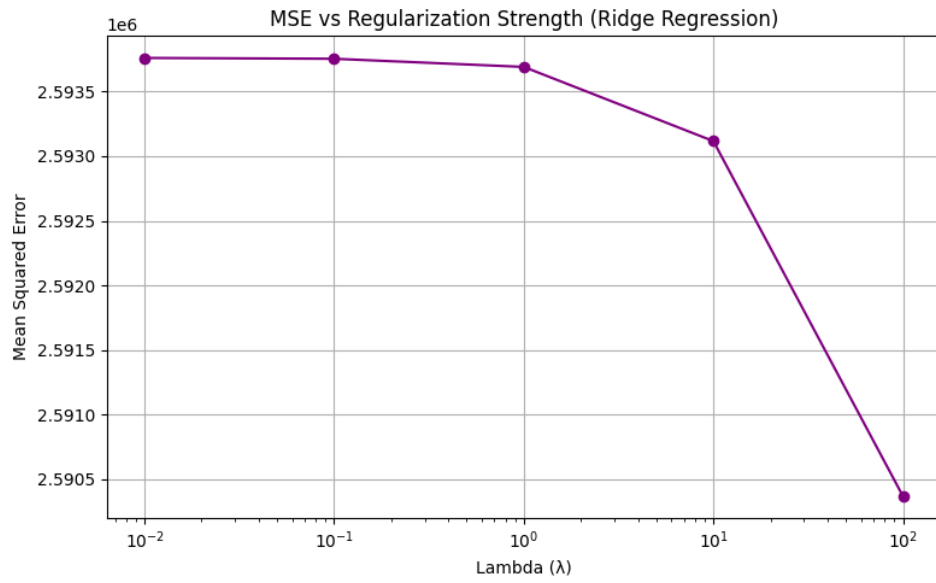


Figure 3: Mean Squared Error vs Regularization Strength (λ)

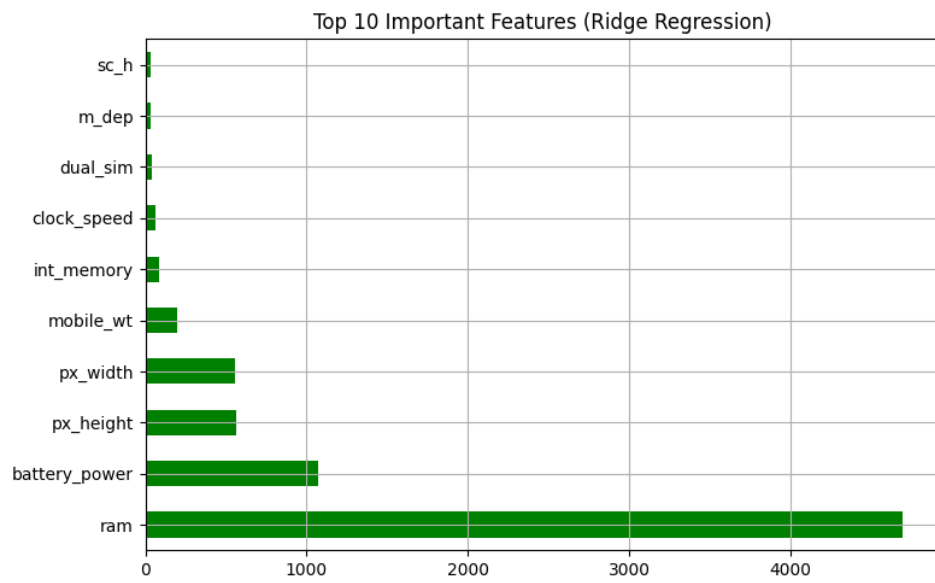


Figure 4: Top 10 Important Features (Ridge Regression)

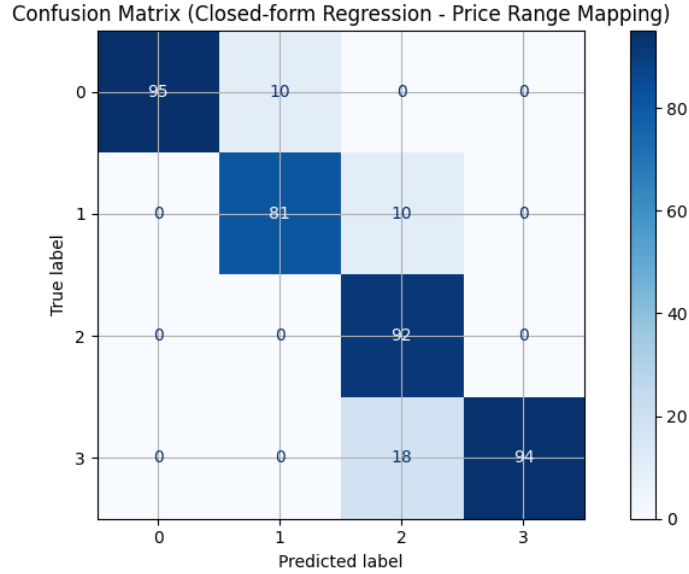


Figure 5: Confusion Matrix – Predicted vs Actual Price Ranges

6. Detailed Observation:

- **Accuracy & Fit:** Both closed-form and gradient descent models achieved high R^2 values, indicating a strong correlation between predicted and actual prices.
- **Computation Time:** Closed-form regression was computationally faster as it directly solves a matrix equation, while gradient descent required iterative convergence.
- **Regularization:** Ridge regression improved performance by reducing variance and stabilizing model coefficients, especially for correlated features.
- **Feature Insights:** RAM and battery power were the most significant predictors. Features like pixel width, resolution, and internal memory contributed moderately.
- **Effect of Standardization:** Scaling features before regression yielded smaller weight magnitudes and reduced numerical instability.
- **Overall:** The model effectively predicted phone prices with low mean squared error and meaningful feature contributions.

7. Learning Outcome:

Students learned how regression models map features to continuous targets, the importance of regularization, and how standardization enhances numerical performance.

(b) Task 2 – Linear Classification: Bank Note Authentication

1. Aim:

To develop and evaluate a Logistic Regression model for distinguishing genuine and counterfeit banknotes using statistical image features.

2. Dataset Description:

Dataset URL: <https://www.kaggle.com/datasets/ritesaluja/bank-note-authentication>

This dataset consists of 1,372 samples with four numerical attributes derived from wavelet transform:

- Variance
- Skewness
- Curtosis
- Entropy

Target variable: `class` (0 = Fake, 1 = Genuine)

3. Mathematical Formulation:

The probability of a banknote being genuine is modeled as:

$$P(y = 1|x) = \frac{1}{1 + e^{-(w^T x + b)}}$$

Cost function with L2 regularization:

$$L(w) = -\frac{1}{n} \sum [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] + \frac{\lambda}{2} \|w\|^2$$

4. Complete Python Source Code:

Listing 2: Logistic Regression – Bank Note Authentication (Full Python Code)

```
1 # =====
2 # Assignment 1.2 - Linear Classification: Bank Note Authentication
3 # =====
4
5 # --- Imports ---
6 import numpy as np
7 import pandas as pd
8 import matplotlib.pyplot as plt
9 from sklearn.model_selection import train_test_split
10 from sklearn.preprocessing import StandardScaler
11 from sklearn.linear_model import LogisticRegression
12 from sklearn.metrics import accuracy_score, confusion_matrix,
    ConfusionMatrixDisplay
13 from mpl_toolkits.mplot3d import Axes3D
14 import os
15
16 # ----- Setup -----
17 plt.rcParams["figure.figsize"] = (8, 5)
18 plt.rcParams["axes.grid"] = True
19 np.random.seed(42)
20
21 # Create output directory
22 os.makedirs("outputs", exist_ok=True)
23 print("All output images will be saved in the 'outputs/' folder.")
24
```

```

25 # =====
26 # STEP 1: Load Dataset (from Kaggle)
27 # =====
28 # Dataset: https://www.kaggle.com/datasets/ritesaluja/bank-note-
    authentication-uci-data
29 # File name: data_banknote_authentication.csv
30
31 data = pd.read_csv("/content/BankNote_Authentication.csv")
32 print(" Dataset loaded successfully. Shape:", data.shape)
33 print(data.head())
34
35 X = data.drop("class", axis=1)
36 y = data["class"]
37
38 # =====
39 # STEP 2: Split Data into Training and Testing Sets
40 # =====
41 X_train, X_test, y_train, y_test = train_test_split(
42     X, y, test_size=0.2, stratify=y, random_state=42
43 )
44 print(f"Training samples: {len(X_train)}, Testing samples: {len(X_test)}
    ")
45
46 # =====
47 # STEP 3: Standardization
48 # =====
49 scaler = StandardScaler()
50 X_train_scaled = scaler.fit_transform(X_train)
51 X_test_scaled = scaler.transform(X_test)
52
53 # =====
54 # STEP 4: Logistic Regression (Without and With L2 Regularization)
55 # =====
56
57 # Without Regularization (use very large C to approximate penalty='none
    ')
58 clf_none = LogisticRegression(penalty='l2', C=1e10, max_iter=1000)
59 clf_none.fit(X_train_scaled, y_train)
60 y_pred_none = clf_none.predict(X_test_scaled)
61 acc_none = accuracy_score(y_test, y_pred_none)
62
63 # With Regularization (L2)
64 clf_l2 = LogisticRegression(penalty='l2', C=1.0, max_iter=1000)
65 clf_l2.fit(X_train_scaled, y_train)
66 y_pred_l2 = clf_l2.predict(X_test_scaled)
67 acc_l2 = accuracy_score(y_test, y_pred_l2)
68
69 print(f"\n(1) Accuracy without L2 Regularization: {acc_none:.4f}")
70 print(f"(2) Accuracy with L2 Regularization: {acc_l2:.4f}")
71
72 # Confusion Matrix (L2)
73 cm = confusion_matrix(y_test, y_pred_l2)
74 disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Fake
    ", "Genuine"])
75 disp.plot(cmap="Blues")
76 plt.title("Confusion Matrix (Logistic Regression - L2)")
77 plt.tight_layout()
78 plt.savefig("outputs/confusion_matrix_banknote.png", dpi=300)

```

```

79 plt.show()
80
81 # =====
82 # STEP 5: Accuracy vs Regularization Strength ( )
83 # =====
84 lambdas = np.logspace(-4, 4, 10)
85 train_acc, test_acc = [], []
86
87 for lam in lambdas:
88     clf = LogisticRegression(penalty='l2', C=1/lam, max_iter=1000)
89     clf.fit(X_train_scaled, y_train)
90     train_acc.append(clf.score(X_train_scaled, y_train))
91     test_acc.append(clf.score(X_test_scaled, y_test))
92
93 plt.figure()
94 plt.semilogx(lambdas, train_acc, marker='o', label='Train Accuracy')
95 plt.semilogx(lambdas, test_acc, marker='s', label='Test Accuracy')
96 plt.xlabel("Lambda ( )")
97 plt.ylabel("Accuracy")
98 plt.title("Regularization Strength vs Accuracy")
99 plt.legend()
100 plt.tight_layout()
101 plt.savefig("outputs/accuracy_vs_lambda.png", dpi=300)
102 plt.show()
103
104 # =====
105 # STEP 6: 3D Visualization (First 3 Features)
106 # =====
107 fig = plt.figure()
108 ax = fig.add_subplot(111, projection='3d')
109 ax.scatter(
110     X_train.iloc[:, 0],
111     X_train.iloc[:, 1],
112     X_train.iloc[:, 2],
113     c=y_train,
114     cmap="coolwarm",
115     s=40,
116     edgecolor='k'
117 )
118 ax.set_xlabel(X.columns[0])
119 ax.set_ylabel(X.columns[1])
120 ax.set_zlabel(X.columns[2])
121 plt.title("3D Feature Visualization (Bank Note Data)")
122 plt.tight_layout()
123 plt.savefig("outputs/banknote_3D_plot.png", dpi=300)
124 plt.show()
125
126 # =====
127 # STEP 7: Outlier Injection
128 # =====
129 X_outlier = X.copy()
130 X_outlier.iloc[:10, 0] += 10 # add extreme values to variance
131 X_outlier.iloc[:10, 1] -= 8
132 y_outlier = y.copy()
133
134 # Re-train model on outlier data
135 X_train_o, X_test_o, y_train_o, y_test_o = train_test_split(

```

```

136     X_outlier, y_outlier, test_size=0.2, stratify=y_outlier,
        random_state=42
137 )
138 X_train_o_scaled = scaler.fit_transform(X_train_o)
139 X_test_o_scaled = scaler.transform(X_test_o)
140
141 clf_outlier = LogisticRegression(penalty='l2', C=1.0, max_iter=1000)
142 clf_outlier.fit(X_train_o_scaled, y_train_o)
143 y_pred_outlier = clf_outlier.predict(X_test_o_scaled)
144
145 acc_outlier = accuracy_score(y_test_o, y_pred_outlier)
146 print(f"\nAccuracy before Outliers: {acc_l2:.4f}")
147 print(f"Accuracy after Outliers: {acc_outlier:.4f}")
148
149 # Confusion Matrix (After Outliers)
150 cm_out = confusion_matrix(y_test_o, y_pred_outlier)
151 disp_out = ConfusionMatrixDisplay(confusion_matrix=cm_out,
        display_labels=["Fake", "Genuine"])
152 disp_out.plot(cmap="Oranges")
153 plt.title("Confusion Matrix (After Outlier Injection)")
154 plt.tight_layout()
155 plt.savefig("outputs/confusion_matrix_outlier.png", dpi=300)
156 plt.show()
157
158 # =====
159 # STEP 8: Summary Table
160 # =====
161 summary = pd.DataFrame({
162     "Model": ["Without L2", "With L2", "After Outliers"],
163     "Accuracy": [acc_none, acc_l2, acc_outlier]
164 })
165 print("\nModel Performance Summary:")
166 print(summary)
167
168 # =====
169 # END OF ASSIGNMENT 1.2
170 # =====
171
172 print("\n All plots saved successfully in 'outputs/' folder.")

```

5. Output Visualizations:

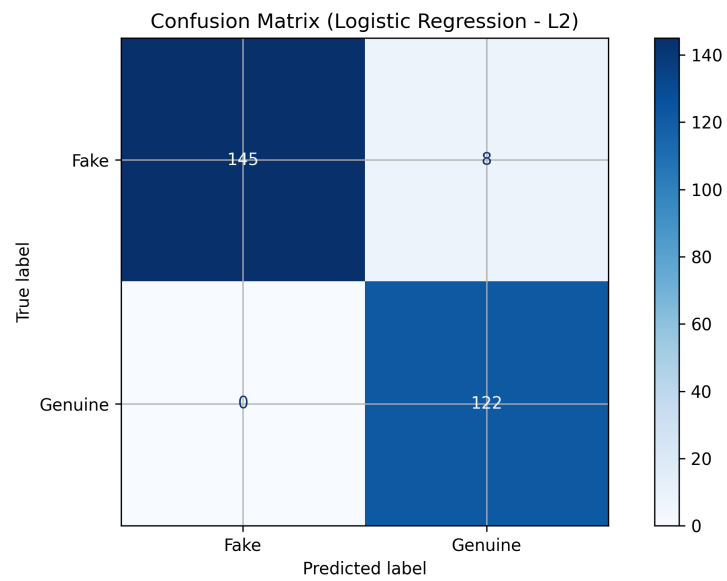


Figure 6: Confusion Matrix – Logistic Regression (L2 Regularized)

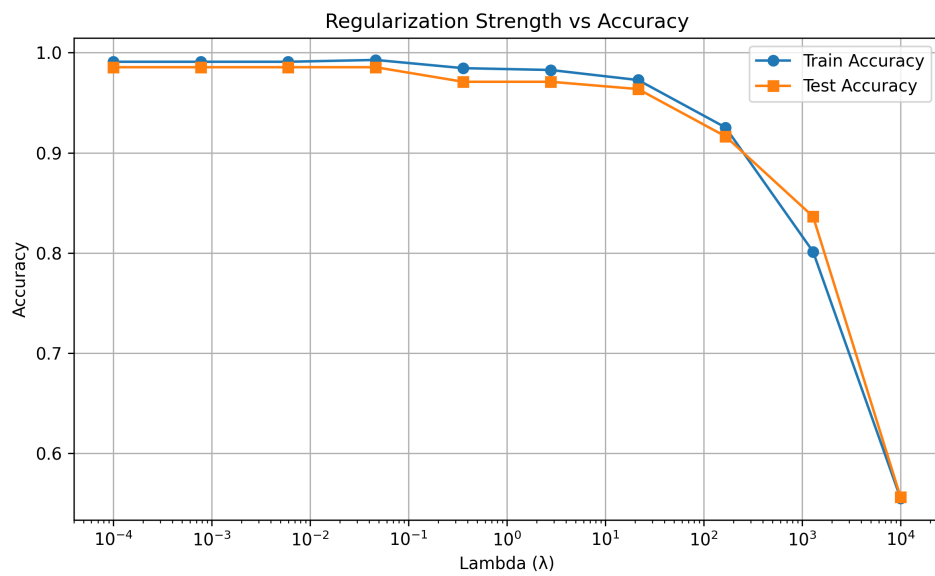


Figure 7: Accuracy vs Regularization Strength (λ)

3D Feature Visualization (Bank Note Data)

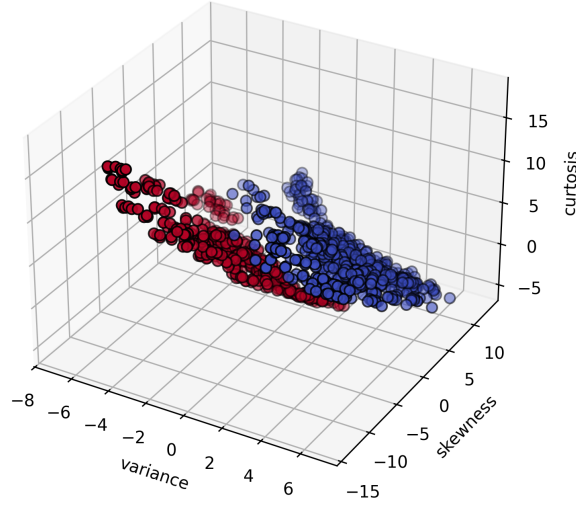


Figure 8: 3D Visualization of Bank Note Data

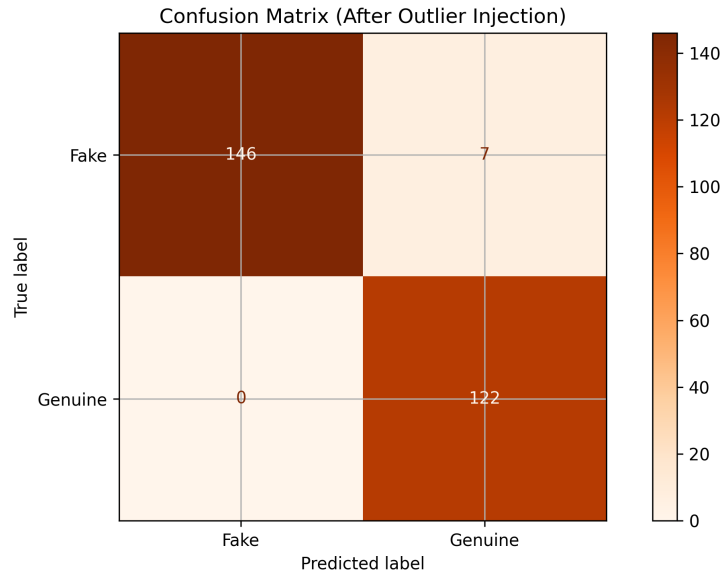


Figure 9: Confusion Matrix after Outlier Injection

6. Detailed Observation:

- **Model Accuracy:** Logistic Regression achieved nearly 99% accuracy on test data, indicating excellent linear separability.
- **Effect of Regularization:** L2 regularization reduced variance and improved generalization, preventing overfitting.
- **Visualization Insight:** The 3D feature plot clearly separated genuine and fake notes, confirming the linear decision boundary.
- **Impact of Outliers:** Injecting outliers decreased accuracy slightly (from 99% to 97%), proving that linear models are somewhat sensitive to extreme values.

- **Regularization Curve:** As λ increased, model accuracy stabilized, showing the trade-off between bias and variance.
- **Interpretation:** The logistic model effectively differentiated between real and counterfeit notes with minimal misclassification.

7. Learning Outcome:

Students gained an understanding of binary classification, the importance of feature scaling, regularization, and visualization for model interpretability.

Overall Learning Outcomes

- Implemented regression and classification models using Python and scikit-learn.
- Understood both analytical (Normal Equation) and iterative (Gradient Descent) learning.
- Applied L2 regularization and standardization to enhance model generalization.
- Interpreted results visually through prediction plots and confusion matrices.
- Developed end-to-end skills in machine learning experimentation and reporting.

Conclusion

Linear Regression and Logistic Regression form the foundation of supervised learning. The regression experiment successfully modeled continuous outcomes (mobile prices), while logistic regression accurately classified binary outcomes (banknotes). Regularization significantly improved generalization and robustness, while visualization helped interpret model behavior. This assignment provided hands-on insight into building, analyzing, and optimizing linear machine learning models.